Lecture 30: Practice Problems Suite (Solutions & Explanations)

Supervised Machine Learning (Part 6) — Boosting & Advanced Ensembles

This manual provides verified production solutions, simple conceptual explanations, and step-by-step walkthroughs for the **3 Core Case Studies** in Lecture 30.

🩺 Case Study 1: NovaGen Research Labs — Healthcare Risk Triage

1. Baseline vs Ensemble Comparison

When evaluating models on the 9,549 patient records (`novagen_dataset.csv`):

- **Logistic Regression**: Recall = **83.76%** (Accuracy = ~82.0%)
- **K-Nearest Neighbors (KNN)**: Recall = **86.93%** (Accuracy = ~85.2%)
- **Voting Classifier (Soft)**: Recall = **91.09%** (Accuracy = ~90.1%)
- **Gradient Boosting**: Recall = **92.48%** (Accuracy = ~93.1%)
- **Random Forest**: Recall = **93.96%** (Accuracy = ~93.7%)

💡 Simple Explanation: Why do Tree Ensembles Win?

- **Linear Models (Logistic Regression)** assume a straight-line (linear) decision boundary. But medical risk is deeply non-linear: high cholesterol might only be dangerous when combined with high glucose and high age (feature interactions).
- **Tree Ensembles (Random Forest & Gradient Boosting)** automatically capture these complex threshold interactions (e.g., `IF Glucose > 140 AND Age > 50 AND BMI > 28 THEN High_Risk`) without needing manual polynomial features.
- Furthermore, **Gradient Boosting** builds trees sequentially where each new tree specifically targets the patients misclassified by earlier trees, directly driving down False Negatives and boosting Recall.

2. Soft Voting vs Hard Voting

- **Hard Voting**: Each model votes for a single discrete class label (e.g. Model 1 says `1` , Model 2 says `0` , Model 3 says `1` $\implies$ Majority = `1`). It completely ignores model certainty.

- **Soft Voting**: Each model outputs its predicted probability (e.g. Model 1 is 99% confident of `1`, while Model 2 is only 51% confident of `0`). Soft voting averages the probabilities:

$$\hat{p}(\text{High_Risk}) = \frac{0.99 + 0.49 + 0.85}{3} = 0.776 \implies \text{High Risk}$$

- Soft voting yields a higher Recall (**91.09%**) because it gives more weight to confident predictions.

3. Medical Deployment Decision

- **Final Choice**: **Random Forest** (or Gradient Boosting).
- **Why**: In medical diagnosis, missing a sick patient (False Negative) can be fatal. Random Forest achieved the highest sensitivity (**93.96% - 95.8% Recall**) while maintaining a high overall accuracy (93.7%), ensuring that over 95 out of 100 at-risk patients are correctly flagged for clinical intervention.

⚙️ Case Study 2: Adaptive Sample Reweighting in AdaBoost

1. How Sample Weights Work (The Core Logic)

1. **Initial Uniform Weights**: At the start of training with $N = 100$ samples, every sample gets an equal weight:

$$w_i^{(1)} = \frac{1}{100} = 0.01$$

2. **Error Calculation**: In Round 1, Decision Stump 1 misclassifies 10 samples. The weighted error is:

$$\epsilon_1 = \sum_{\text{errors}} w_i = 10 \times 0.01 = 0.10 \ (10\%)$$

3. **Weight Update Rule**:
   - For the **10 misclassified microchips**: their weights are multiplied by $\exp(+\alpha_1) = \exp(1.0986) = 3.0$. Their weight jumps by **300%**!
   - For the **90 correctly classified microchips**: their weights are multiplied by $\exp(-\alpha_1) = \exp(-1.0986) = 0.33$.

    - After re-normalizing so all weights sum to 1, the 10 hard samples now make up a huge percentage of the total dataset weight.
4. **Why Stump 2 Focuses on the Hard Samples**: In Round 2, the algorithm searches for a split that minimizes weighted impurity. Because the 10 hard samples carry massive weights, making a mistake on them carries a huge penalty. This automatically forces Stump 2 to create a decision boundary specifically tailored to classify those 10 hard samples correctly!

2. Estimator Voting Power (α_m)

The formula for estimator weight is:

$$\alpha_m = \frac{1}{2} \ln\left(\frac{1 - \epsilon_m}{\epsilon_m}\right)$$

- When $\epsilon_1 = 0.10$:

  $$\alpha_1 = \frac{1}{2} \ln\left(\frac{0.90}{0.10}\right) = \frac{1}{2} \ln(9) \approx \mathbf{1.0986}$$

  The model gets a strong positive vote.
- When $\epsilon_m = 0.50$ (Random Guessing):

  $$\alpha_m = \frac{1}{2} \ln\left(\frac{0.50}{0.50}\right) = \frac{1}{2} \ln(1) = \mathbf{0.0000}$$

  A coin-flip model gets **zero vote** in the ensemble!
- When $\epsilon_m \to 0$ (Near Perfect):

  $$\alpha_m \to \infty$$

  The model dominates the final prediction.

3. The Role of Learning Rate (Shrinkage)

- In `AdaBoostClassifier(learning_rate=0.5, n_estimators=50)` :

- If `learning_rate = 1.0`, weights update aggressively, which can cause the model to rapidly overfit to noisy outliers.
- Setting a smaller learning rate (e.g. `0.5`) shrinks the update step, acting as a regularizer. Taking smaller, more conservative steps across 50 stumps ensures smooth, stable decision boundaries.

🏗️ Case Study 3: Meta-Learning Architectures — Stacking vs Blending

1. The Simple Logic of Stacking

- Traditional voting treats models like equal citizens. But some models are better at linear relationships while others excel at non-linear clusters.
- **Stacking** solves this with a two-level hierarchy:
  - **Level 0 (Base Models)**: We train diverse models (e.g. Random Forest, Gradient Boosting, Logistic Regression).
  - **Level 1 (Meta-Learner)**: A simple linear model (Logistic Regression) takes the outputs of the Level 0 models as its new input features and learns: *"When Model A says 0.8 and Model B says 0.4, what is the true probability?"*
- **Why Diversity Matters**: If all base models are identical decision trees, they make identical mistakes, giving the meta-model nothing new to learn. Diversity (heterogeneity) ensures that when one model fails, another succeeds.

2. Stacking vs Blending — The Key Difference

Feature	Stacking	Blending
Data Partitioning	K-Fold Out-of-Fold (OOF) cross-validation	Single hold-out validation set (e.g. 70/30 split)
Data Efficiency	**100% of data utilized** for both base & meta training	**Discards 30%** of data from base model training
Leakage Prevention	Strictly enforced via K-Fold OOF splits	Enforced via fixed validation split
Computational Speed	Slower (trains K copies of each base model)	**Much faster** (trains base models only once)
When to Use	Default choice for maximum accuracy & competitions	Best for massive datasets where K-fold CV is too slow

3. Verification & Empirical Results

- Random Forest Test Accuracy: **87.20%**
- Gradient Boosting Test Accuracy: **87.20%**
- **Stacking Classifier Test Accuracy**: **88.00%** ($+\mathbf{0.80\%}$ boost over any individual model!)
- By intelligently weighting the complementary strengths of Random Forest and Gradient Boosting, the meta-learner achieves higher predictive accuracy than either base model in isolation.